

Metric Handbook

Matthew Santorsa

Table of Contents

Metric Handbook.....	3
Definitions.....	3
Worked Example.....	4
Templates.....	4
Matrix Orientation.....	4
Experiment IDs.....	5
Evidence Map.....	5
Graph Templates.....	5
Manuscript Outline.....	5

Metric Handbook

Definitions

- Exhaustive Test Generation: The process of applying every possible input combination (test vector) to a circuit to guarantee that all detectable faults are observed. For an n -input circuit, this requires 2^n test vectors.
- Fault-Detection Matrices: A 2D grid mapping test vectors to the specific faults they detect. Rows represent test vectors, columns represent potential faults, and a cell contains a 1 if the vector detects the fault while a 0 if not.
- Fault Coverage: Representation of the proportion of total possible faults that are successfully detected by a given set of test vectors. It is calculated as:
$$\text{Coverage} = (\text{Detected Faults} / \text{Total Faults}) \times 100 = 100\%$$
- Set-Cover Interpretation: Framing the testing process as a mathematical set-cover problem, where the universe is the total set of faults, and each test vector represents a subset of detectable faults. The goal is to find the minimum number of subsets that completely cover the universe.
- Greedy Selection: Heuristic optimization algorithm that builds a solution step-by-step by always choosing the next test vector that will detect the largest number of previously uncovered faults, without reconsidering past choices.
- Test-Set Reduction: The percentage decrease in the number of required test vectors compared to an exhaustive test set. Calculated as: $\text{Reduction} = (1 - (\text{Selected Vectors} / \text{Total Vectors})) * 100$.

- Worked Example Math: In the AND gate example below, the greedy algorithm selects 1 vector out of 4 total possible vectors. Reduction = $(1 - (1 / 4)) * 100 = 75\%$.
- Worked Example Fault Coverage Math: The chosen vector successfully detects the 1 injected fault. Coverage = $(1 / 1) * 100 = 100\%$.

Worked Example

Below is a table depicting a 2 input AND gate, which shows faulty versus correct outputs to help identify successful test vectors. The setup is 2 inputs, A and B, with 1 output Z. The injected fault is a stuck at 1 fault, which implies the gate will always behave as if input A is receiving a 1.

Test Vector (a,b)	Correct Output (z)	Faulty Output (Z_faulty)	Fault Detected?
V1: (0,0)	0	0	No
V2: (0,1)	0	1	Yes (inconsistent outputs)
V3: (1,0)	0	0	No
V4: (1,1)	1	0	No

We can conclude that the only successful vector is vector 2, as it is the only vector capable of showing the fault altering logic output

Templates

Matrix Orientation

- The detection matrix utilizes a sparse dictionary (hash map) data structure.
- Rows (Keys): Immutable input tuples representing binary test vectors
- e.g., (0, 1, 1)
- Columns (Values): Mathematical sets containing string-based fault identifiers
- e.g., {"A_SA1", "B_SA0"}

Experiment IDs

- Files and runs will use the standardized naming convention: [CircuitName]_[FaultType]_[RunDate]
- Example: Adder_StuckAt_10242026.json

Evidence Map

- Generation: Python exhaustive generator creates binary tuples.
- Simulation: Fault engine runs tuples and outputs detected string ids.
- Storage: Detection-matrix module parses output into the sparse dictionary container.
- Optimization: Greedy selector algorithm ingests the matrix and outputs a reduced vector tuple list.
- Reporting: Metric functions process the lists and export JSON/CSV data for manuscript tables.

Graph Templates

- Coverage vs. Reduction: Scatter plot mapping fault coverage percentage (y-axis) against test-set

reduction percentage (x-axis) across all benchmark circuits.

- Runtime Performance: Bar chart comparing exhaustive simulation execution time versus greedy selection execution time (in milliseconds) per circuit.

Manuscript Outline

I. Abstract

A. Summary of the reversible fault-detection problem, the proposed greedy optimization method, and primary coverage/reduction findings.

II. Introduction

A. Importance of reversible circuit testing, quantum cost considerations, and problem statement.

III. Related Work

A. Prior methodologies in fault detection, simulation, and set-cover heuristic applications.

IV. Methodology

A. Simulator & Fault Engine: Interface rules and fault generation semantics.

B. Detection-Matrix Architecture: Data schemas and integration.

C. Optimization Algorithm: Deterministic greedy selection and index-based tie-breaking rules.

V. Experiments and Results

A. Baseline Analysis: Exhaustive matrix data for primary circuits.

B. Optimization Outcomes: Fault coverage, test-set reduction percentages, and execution runtimes.

VI. Discussion and Threats to Validity

A. Interpretation of algorithmic efficiency, limitations of heuristic sub-optimality, and external benchmark applicability.

VII. Conclusion

A. Final statement on method viability and future research paths.